Jet Engine Anomaly Detection Scoring Using NASA N-CMAPSS

Luciana Qu, Danielle Vykydal, & Dillon Wollberg

Introduction

Engine degradation usually shows up first as small shifts in temperatures, pressures, and speeds, long before anything breaks. Most machine learning work on NASA's C-MAPSS and N-CMAPPS data predicts remaining useful life (RUL), which counts down to failure. We wanted to ask an earlier question: can we tell when an engine stops behaving like a healthy one? Our goal was early detection that would prompt inspection, not replacing manufacturer maintenance schedules. If this problem were fully solved ,catching a malfunction wouldn't depend on one sensor crossing a limit. All of an engine's sensors would be watched together, so that subtle, system-wide changes could be caught before they turned into failures. Two things motivated us. Firstly, N-CMAPSS is large and realistic; our subset alone had 8.6 million rows recorded at 1Hz over real flight profiles. Secondly, manufacturing testing showed why early detection mattered. Faulty equipment tended to fail either immediately or within an initial period; and RUL-focused work rarely looked at that early stage. We approached the problem in two ways, both using NASA's DS08a subset (15 engines, 994 flights) and splitting the data by engine.

- **Part A (Supervised):** Tree based ensembles were used to identify the most important features. It trained on health and unhealthy engine data to learn the boundary between health and unhealthy. It then predicts the healthy data. Feature importance analysis was then used to carry results over to the multi-layer perception (MLP) portion. MLP supervised learning trained on data screened for healthy status. It used the top features from the tree based analysis as a prediction target. Anomaly detection was performed on regression error to classify unhealthy engines.
- **Part B (Unsupervised):** Both models were trained only on healthy flights and never saw health labels. Before modeling, we removed the effect of flight conditions (altitude, Mach, throttle, and inlet temperature) from each sensor with a regression fit on healthy data, so what remained was the engine's departure from healthy behavior. An Isolation Forest scored each flight by how easily it could be isolated from healthy flights. An LSTM autoencoder scored 10-step sliding windows by how poorly it could reconstruct them. Health labels were used only to evaluate the scores.

Compared with related work, our project contributes three things:

1. We evaluate against the simulator's true health labels on every flight, including the transition into degradation, rather than a heuristic healthy cutoff.
2. We compare a tree-based and a sequence-based detector on the same engines with the same kind of correction.
3. We test whether the choice of correction model (gradient boosting vs. MLP) changes the results.

Main Findings: Both parts arrived at the same lesson from different directions: features mattered more than models. Supervised: Features were more important than the models, and the top features were related to HPT performance. In Part B, removing operating-condition effects took the Isolation Forest from near chance on raw sensors (test ROC-AUC 0.64) to 0.81, while its own hyperparameters moved validation ROC-AUC by less than 0.01. Our final LSTM autoencoder achieved an overall ROC-AUC score of around 0.80 and a PR-AUC score of about 0.90.

Related Work

Most work on C-MAPSS and N-CMAPSS predicts remaining useful life (RUL) (e.g., Xu et al., 2023). Fewer studies ask whether an engine is still healthy *right now*. The four studies below are the closest to our project. It is also worth noting that our milestone is not an extension of a previous course project.

1. Cohen, Huan, & Ni (2023) pooled all 90 N-CMAPSS engines and trained tree ensembles and neural networks on PCA-reduced, per-flight features. Their models predicted health state (`hs`), which components would eventually fail, and RUL, with ROC-AUC and PR-AUC above 0.95.
   a. **How Ours Differs:** Part A also classifies `hs`, but on DS08a alone, where all five components degrade together. In Part B, `hs` is used only for evaluation, never for training.
2. Sánchez et al. (2026) trained an LSTM autoencoder on classic C-MAPSS, treating the first 85% of each engine's life as healthy. They removed operating-condition effects with an MLP regression fit on healthy data.
   a. **How Ours Differs:** N-CMAPSS has true health labels, so we don't need a heuristic healthy cutoff; and its real flight profiles have f operating variables instead of three. We followed their correction idea, added an Isolation Forest alongside the autoencoder, and tested whether the choice of correction model mattered (i.e., gradient boosting vs. MLP)
3. Al Bataineh, Mairaj, & Kaur (2020) trained an autoencoder on the first 60% of each C-MAPSS FD004 engine's life. They tested only the first 60% and the last 5% of each life, and reached an F1 score of 0.892. Jakubowski et al. (2022) similarly dropped intermediate cycles when benchmarking on FD004.

   a. **How Ours Differs:** We scored every flight, including the transition where degradation begins, and reported ROC-AUC, PR-AUC, and detection delay. That makes the task harder, but it's closer to how an early-warning system would actually be used.

4. Michau & Fink (2020) showed that anomaly detectors trained on healthy data raise false alarms under operating conditions they haven't seen. They addressed this with adversarial domain adaptation on simulated turbofan flights.
   a. **How Ours Differs:** We encountered the same problem. On raw sensors, our Isolation Forest mostly tracked altitude and throttle (test ROC-AUC 0.64). A simpler per-sensor regression residual, fit on healthy training flights, raised that to 0.82.

Data Source(s)

Dataset number seventeen: Turbofan Engine Degradation Simulation-2 (N-CMAPSS) from NASA's Prognostics Center of Excellence dataset repository. The data comes in a ZIP file of several .h5 datasets. There is a paper "Run_to_Failure_Simulation_Under_Real_Flight_Conditions_Dataset" with tables and illustrations that explain some of the datasets. There is a .py file "N-CMAPSS_Example_data_loading_and_exploration" which was referenced in the extraction and feature engineering of the datasets. Ultimately, a different approach was taken to assemble the dataset.

- DS08a was used in this study. It was the most feature dense file that would fully convert out of the .h5 format.
- All sensor sampling rates were at 1Hz, one cycle per second (Arias Chao et al., 2021). Additionally, all datasets had the same features but many were sparse because the experimental setup did not always require a full sampling of datapoints.
    - For example: not all trials measured fan section degradation, so the fan stall margin feature was populated with zeros.
- DS08a was fully populated with about 8.6 million rows and 45 columns. Some columns were duplicate or not helpful (duplicated unit and cycle columns). Other features will be covered in the next section.

Feature Engineering

Before deciding which dataset(s), we loaded them into a postgreSQL database (hosted on Neon). We explored the tables and found how sparse some of the datasets were. Not all sections were represented in each dataset. Engine unit numbers did not have continuity across datasets, meaning that unit one was not the same engine in dataset one and two. Additionally, the datasets were poorly structured for a relational database. It was sampled by the second, with no timestamps. Following Arias Chao et al. (2021), we built a separate table for each dataset.

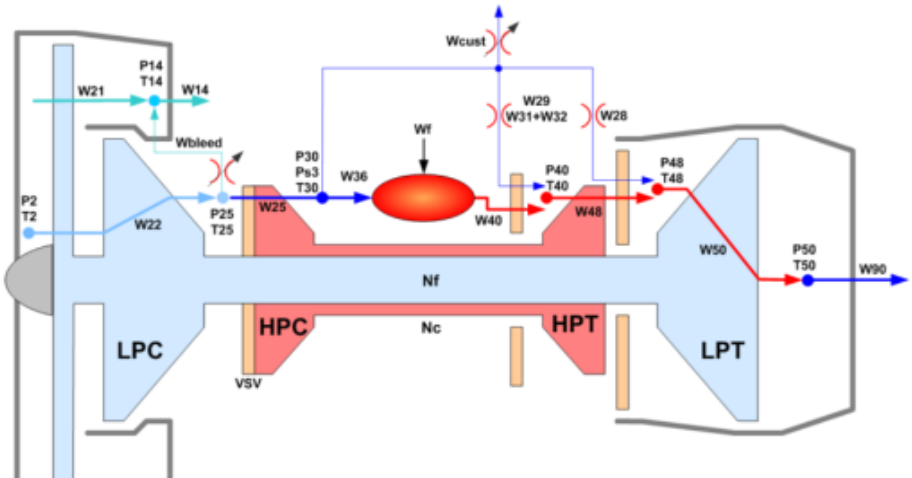
The DS08a dataset was migrated from PostgreSQL database to a CSV. With three different flight classes (fc) (flight durations). Short flights had 1.7 million samples. It was downsampled to every 76sec producing about 22,368 samples. Medium duration flights had 2.8 million samples. downsampled to every 127sec producing about 22,047 samples. Long duration flights had 4 million samples. downsampled to every 180sec producing about 22,222 samples.

Columns for the supervised machine learning models were renamed so they would be easier to read. Provided efficiency parameters were kept. Their naming conventions were (section)_*(flow or eff)*_mod. Arias Chao et al. (2021) used these prognostic health parameters for remaining useful life (RUL) estimation. They were used in the logistic regression multi-layer perceptron (MLP) as a predictor to assess anomaly detection against other engineered features illustrated in in table 1(N-CMAPSS Supervised MLP Efficiency Metrics).

We also added 'inlet_maf', the inlet mass airflow. To calculate the fan input airflow, the ambiance library was used for air density and speed (mach) at altitude. $mass_airflow = m_dot = rVA$ (National Aeronautics and Space Administration [NASA], n.d.) Other engineered efficiency metrics were simply the ratio of output to input for flow, temperature, or pressure (as available).

Table 2 in the appendix is the column name mapping from sensor to functional description.

Figure 1 (Arias Chao et al., 2021) labels the sensor locations.



Part A. Supervised Learning (35 Total)

Methods Description (8 points)

The supervised learning workflow was in two parts. Each part had their own preprocessing, models, and evaluation portions. Each compared at least three models. Both split the data by engine unit.

Part 1: *Tree Ensembles*. To predict the health status of the engine units, and to find the most informative features for such predictions. Tree-based models

(XGBoost, LightGBM, and CatBoost) were chosen because they need little tuning. As powerful models, they could learn non-linear relationships and extract feature importance. With an upper limit of 2000 estimators, early stopping chose the best count. SHAP value ranked the features (Figure A1).

Part 2: *Feed Forward Neural Networks*. Trained on healthy engine units, this anomaly detection was implemented using the most important features prescribed by the tree ensemble. Unhealthy units were flagged as anomalies with a threshold of the 95th percentile of training error.

There were six networks: Logistic Regression, SmallMLP, MediumMLP, LargeMLP, DropoutMLP, and WideMLP. A training class fit each one under each of two loss functions (MSELoss and HuberLoss). AdamW, learning rate, and epochs were class arguments. We did not record per-epoch results to find the most efficient count because of the time and volume of output. Up to 1000 epochs 6 networks and two loss functions made the output excessive. The DropoutMLP never performed well, and the LargeMLP nearly matched the Medium MLP. All three were dropped.

Features. Each engine section's output is the input for the next. In terms of sensor temperature, pressure, or flow (as available) each section had two efficiency metrics calculated and used to predict the unhealthy state of the engine.

SHAP scores balance out the positive and negative impacts, a mean score of 0 means that the feature explains nothing. Table A1 (in the appendix) lists the metrics with highest SHAP scores to represent different aspects of engine performance per section. [Figure A1 in the appendix is the complete SHAP beeswarm figure.]

Figure A2: `hpt_temp_eff`

```
Metric: hpt_temp_eff
Threshold (95th pct of training error): 0.0107
Accuracy: 0.8103
predicted     0     1
actual
0          4367   685
1           850  2188
```

Figure A2 shows the results of Logistic Regression MLP. hpt_tem_eff being the feature with the top SHAP score.

Supervised Evaluation: Overall Results Reporting

Trees. Mean PR_AUC across CV splits was the primary metric because health_status classes are unbalanced. ROC AUC was used for comparison. Table A2 contains all tree method results, XGBoost performing the best.

Table A2: Model Performance

Model	Trees	Mean CV PR_AUC	CV PR_AUC Std Dev	Test PR_AUC	Test ROC_AUC
XGBoost	553	0.8995	0.0020	0.8499	0.9260
LightGBM	124	0.8679	0.0021	0.7684	0.8914
CatBoost	312	0.8651	0.0027	0.8046	0.9044

Neural networks. With 20 epochs (first training runs) the best model was SmallMLP. Reaching about 37% accuracy for the top feature, the best activation function being CELU. At 1000 epochs, Logistic Regression was the best with 81% accuracy. Huber loss was the best loss function.
Table 3 contains the summary metrics of three trials. Trial 1: hpt_temp_eff with the _mod health parameters and RUL dropped. Trial 2: hpt_temp_eff with _mod features included. Trial 3: HPT_flow_mod as the target feature.
Mean squared error (MSE) and mean absolute error (MAE) are reported because HuberLoss combines them. Quadratic for small errors and linear for large ones; HuberLoss is less sensitive to outliers than MSE.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \quad \text{and} \quad MAE = \frac{\sum_{i=1}^{n}|y_i - x_i|}{n}$$

, where $\hat{y}$ is the predicted value, MSE is used in Huber Loss and MSELoss.

Table A3: MLP Model Loss

Trial	Target Column	Criterion	Model	Train Loss	Test Criterion Loss	Test MSE	Test MAE

1	hpt_temp_eff (_mod health parameters dropped)	HuberLoss	LogisticRegression	0.000021	0.000032	0.000064	0.006347
2	hpt_temp_eff	MSELoss	LogisticRegression	0.000038	0.000050	0.000050	0.005551
2	hpt_temp_eff	HuberLoss	LogisticRegression	0.000019	0.000025	0.000050	0.005551
3	HPT_flow_mod	MSELoss	LogisticRegression	0.113661	1.231500	1.231500	0.867819
3	HPT_flow_mod	HuberLoss	LogisticRegression	0.056759	0.506155	1.229551	0.866868

In-Depth Evaluation

1. Feature Importance & Ablation Analysis

SHAP was used because sensor readings depend on upstream and downstream sections. For example, engine bleed air from the LPC cools other sections. When HPT Temperature rises, the LPT receives less dense air. As a result, LPC is driven less powerfully by the LPT. The throttle resolver angle may need to be adjusted to compensate for the extra fuel demand to keep up with other efficiency losses.

The _mod features dominated feature SHAP values and made efficiency metrics redundant, so they were ablated. Figures A3 and A4 compare how feature importance changed after ablation. Figures A5 and A6 show accuracy improved about 17% after _mod features were dropped.

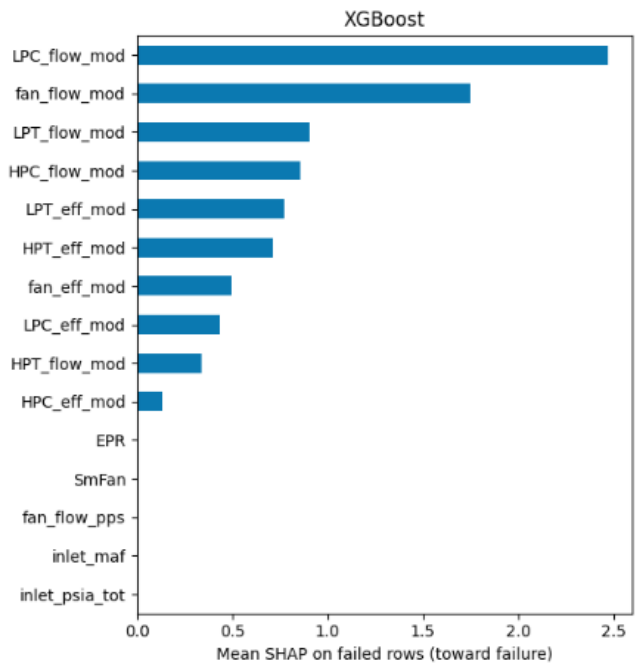


Figure A3: SHAP with _mod features

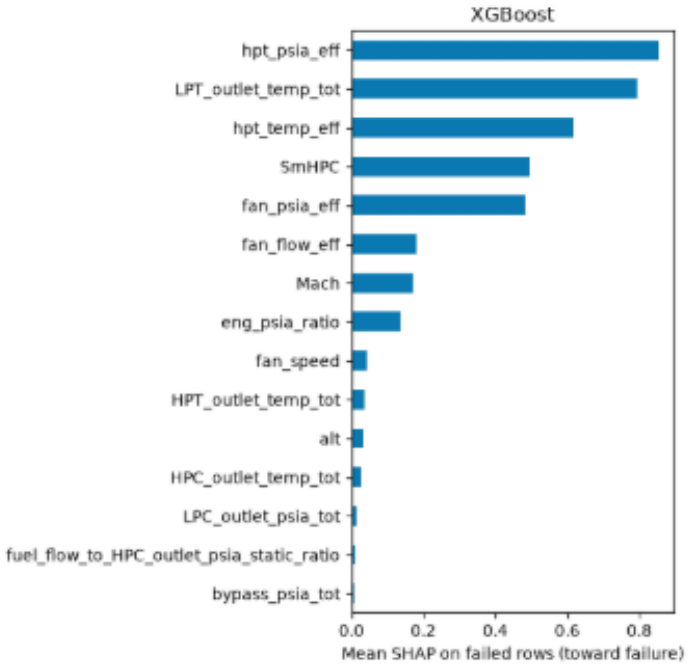


Figure A4: SHAP without _mod features

```
Metric: hpt_temp_eff
Threshold (95th pct of training error): 0.0128
Accuracy: 0.6366
predicted     0     1
actual
0          2392  2660
1           280  2758
```

Figure A5: With _mod features

```
Metric: hpt_temp_eff
Threshold (95th pct of training error): 0.0107
Accuracy: 0.8103
predicted     0     1
actual
0          4367   685
1           850  2188
```

Figure A6: _mod features dropped

2. Sensitivity Analysis

Huber Loss outperformed MSELoss (Table A3). The model was not particularly sensitive to the choice of loss function, considering the fact that MSELoss did not break the model.

The choice of epochs was more impactful. From 20 epochs to 1000, the accuracy improved from about 37% to 81% across 20, 200, 500, and 1000 epochs. The optimal number is likely less than 1000. Because of earlier comments about output size, epoch count was not tuned. It is an optimization problem at that point, not a sensitivity one.

3. Identify Tradeoffs

Trees: PR_AUC scored lower than ROC_AUC (Table A2). However, the mean cv PR_AUC suggests that the gap may not be as wide as initially scored. PR_AUC is the more reliable metric because health_status labels are highly imbalanced.

Neural Networks: The logistic regression traded processing speed, over many epochs, for accuracy.

Failure Analysis

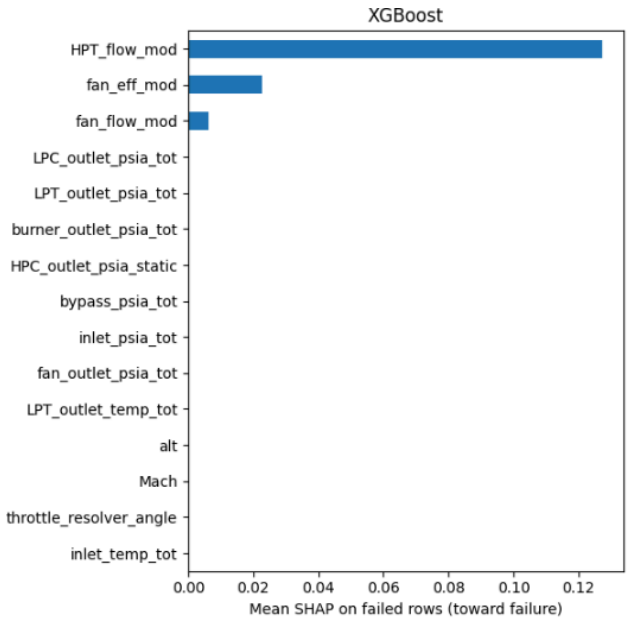
1. ***RUL Data Leakage*:** After the _mod columns were restored, the model overfit to RUL, which is expected because they were underlying parameters. The dominance of HPT_flow_mod is unexpected because there are nine other _mod features.

   After dropping them, the PR_AUC remained at 0.9993 and ROC_AUC at 0.9571. There are almost certainly other features leaking to RUL. Fortunately, the health_status was the outcome variable for the project.

2. ***Group Leakage:*** The column for group labels was dropped and each group leaked between the training and test datasets. A StratifiedGroupKFold split was done in response to keep the groups intact and balance the distribution of groups amongst the datasets.

3. ***Residual Bias:*** Flight classes were not stratified. They contained different representations of the number of cycles and flight durations it takes to reach a failed state. A future split would stratify on features, avoiding bias.

Figure A7: HPT_flow_mod



Part B. Unsupervised Learning

Methods Description

Part B asks whether a model that has only ever seen **healthy** engine data can produce a continuous score that flags degraded/unhealthy engines/flights, without using health labels to train.

We tested two methods with very different mechanisms: an Isolation Forest (IF), which is tree-based; and a recurrent Long Short-Term Memory (LSTM) autoencoder, which is non-probabilistic, reconstruction-based deep learning.

We chose an Isolation Forest as our first method for three reasons. First, it was built for anomaly detection: rather than modeling what “normal” looks like, it would isolate points that are few and different (Liu et al., 2008), which fit a setup where we train only on healthy flights. Second, it made no assumptions about how the data are distributed. Because each split is drawn within a single feature’s range, it also doesn’t need features on the same scale, so it works directly on sensor residuals with very different units. Third, IF is relatively fast and has few hyperparameters, which ultimately allowed us to run a 192-configuration grid with 5 seeds on a laptop. It also gives a tree-based, per-flight contrast to the LSTM autoencoder’s sequence-based reconstruction approach. Its main limitation is that it scores each flight on its own and can’t see trends over time. We addressed that with a rolling median over consecutive flights.

Our LSTM Autoencoder is designed to detect turbofan engine degradation without relying on engine health status labels during training. By referencing similar work done by Sánchez et al. (2026), we built our autoencoder model to compress sequential engine data down into a bottleneck latent vector before expanding it back out through a progressive funnel compression encoder followed by a mirror-expansion decoder, each consisting of three stacked LSTM layers with dropout layers in between.

The autoencoder reads in temporal sliding windows of operating-condition normalized sensor residuals to learn degradation patterns and trajectories over time. Since the training dataset consists exclusively of perfectly healthy engines, the autoencoder learns to reconstruct only healthy temporal dynamics. The reconstruction error, defined here as the Cumulative Mean Absolute Error (MAE) across sensors for each sliding window, is used as a continuous anomaly

score. It is expected that sliding windows containing healthy, non-degraded engines will yield lower cumulative reconstruction errors. On the other hand, is it expected that as an engine approaches its end-of-life and becomes more degraded or unhealthy, the autoencoder will increasingly struggle to reconstruct the sliding window's sensor residuals, resulting in significantly higher cumulative reconstruction errors.

We justify our use of an LSTM autoencoder method for anomaly detection on the N-CMAPSS dataset for several reasons. For one, LSTM-based autoencoders are extensively documented and validated in academic literature as being useful for time-series anomaly detection tasks, with Sánchez et al.'s (2026) work on the original CMAPSS data being a prime example. Autoencoders provide a robust unsupervised method particularly efficient at anomaly detection through their ability to learn how to compress and reconstruct standard or normal operating states. Furthermore, by processing our data as sliding windows instead of single time-steps, our LSTM layers can efficiently detect sensor behavioral patterns and trends over time.

General Workflow of Unsupervised Learning Methods

1. **Data**: We used the DS08a subset from NASA's N-CMAPSS data (`N-CMAPSS_DS08a-009.h5`), downsampled to one row per 76s (flight class or `'Fc'== 1`), 127s (`'Fc'== 2`), and 180s (`'Fc'== 3`); for a total of 67,328 rows, 994 flights, and 15 engines.
2. **Preprocessing:**
   - **Splitting by Engine**: We split the engines for the test, validation, and test splits, with 9 units for training, 3 units for validation, and 3 units for testing (#1, #11, #12). Each set holds an engine belonging to `'Fc'== 1`, `'Fc'== 2`, and `'Fc'== 3`, and zero engines appear in more than one split.
   - **Operating-Condition Regression Normalization:** Both models involve regressing out effects of operating conditions (i.e., altitude `alt`, `Mach` number, throttle angle `TRA`, and inlet temperature `T2`) on raw sensor data in order to isolate true engine degradation.
   - **Residual Scaling (Autoencoder Only):** Sensor residuals are standardized to a uniform scale to prevent unequal treatment of features and effects on the loss function.
   - **Temporal Sliding Windows (Autoencoder Only):** Continuous time-series sensor residuals are transformed into overlapping sequential windows to provide the LSTM layers with temporal context.
3. **Training**: Both models are fitted only on the healthy flights (`hs = 1`) from the training engines: 187 flight summaries for the Isolation Forest, and 10-step window sequences of the same healthy rows (12,518 rows) for the autoencoder. Health labels are used only to evaluate validation/test sets and pick training data as well as validation data (for early stopping with the autoencoder).
   - **Early Stopping (Autoencoder Only):** To prevent overfitting, MAE loss was computed on healthy validation data during training to halt model optimization early if it failed to improve over several epochs
4. **Rolling Smoothing:** We smoothed our anomaly scores via a rolling median window to limit operational noise effects
5. **Scoring**: We scored every flight (Isolation Forest) or sliding window end step (LSTM autoencoder) in the validation and test engines. Higher scores meant greater departure from healthy behavior.
6. **Select on Validation, Report on Test**: Each hyperparameter choice was made on the validation engines only. The test engines were scored only once at the end.

IF: Feature Representation

Each flight was summarized as a mean of each sensor over the flight. Our first Isolation Forest used the raw means of 12 physical sensors (with P21 and P40 dropped as near-duplicates of P15 and Ps30, where $|r| > 0.999$). This barely edged out chance probability (validation ROC-AUC of 0.57), and its performance swung by flight class. Additionally, the FC3 validation engine scored 0.37, which was below random guessing.

The reason turned out to be physics. Operating conditions explain more than 99.9% of each sensor's variation in healthy flights, so the forest was mostly isolating *unusual flights*, not *unhealthy engines*. This was a turning point in our research. We then began building a second representation, operating-condition residuals. For each sensor, a gradient-boosted regressor learns what a healthy engine reads under each flight condition, fit only on healthy rows from the training engines:

$$residual = actual\ sensor\ value - \hat{f}_{healthy}(alt,\ Mach,\ TRA,\ T2)$$

In this sense, a healthy engine's residuals would sit near zero; and a degrading engine would drift away. We then added the 14 virtual sensors (i.e., simulator-calculated flows and stall margins), corrected the same way, for a total of 26 features. Because virtual sensors are not physical measurements, we report results with and without them.

An Isolation Forest (IF) builds many random trees, each splitting on a random feature at a random value until every flight sits alone in a leaf. Unusual flights are isolated in fewer splits, so the anomaly score comes from the average path length across the trees:

$$s(x) = 2^{-E[h(x)]/c(n)},$$

where $E[h(x)]$ is the average path length and $c(n)$ is the expected path length for n training samples. Scores fall between 0 and 1. Because `scikit-learn` returns the negative, we use `-score_samples()`.

IF: Hyperparameter Tuning & Exploration

We tuned in two stages using the validation engines.

1. Feature Construction (one change at a time, Table B1): Starting from the **Raw-Sensor Baseline (Model #1)**, we changed one thing per model and kept whatever improved validation ROC-AUC. We fixed `n_estimators=200` and `random_state=42` throughout.
    a. **Model #2 - 1 Hz Check:** We reran the baseline on the full 1 Hz data to confirm downsampling didn't cost information (0.550 vs. 0.572)
    b. **Model #3 - Operating -Condition Residuals:** We replaced raw sensor meas with residuals. This was the largest single gain (0.572 → 0.848)
    c. **Model #4 - 5-Flight Rolling Median:** We smoothed scores over consecutive flights (0.897).
    d. **Model #5 - Virtual Sensors:** We added the 14 corrected virtual sensors (0.893).
    e. **Model #6 - Both #4 and #5:** Virtual sensors plus smoothing gave the best validation ROC-AUC (0.923). We carried this pipeline into the grid search.
2. Grid search on the final pipeline: 192 configurations ($3 \times 4 \times 4 \times 4 = 192$), each repeated with 5 random seeds. This was because we only had 3 validation engines, so a 0.01-0.02 difference could be due to chance:
    a. **`n_estimators`**: 100, 200, 500
    b. **`max_samples`**: 32, 64, 128, auto
    c. **`max_features`**: 0.25, 0.50, 0.75, 1.0
    d. **Smoothing Window** (rolling median, in flights; a post-processing step, *not* an IF parameter): 1, 3, 5, 10, flights

The best mean validation ROC-AUC was 0.940, and 28 configurations fell within one seed SD of it. Among these, we chose the simplest: 100 trees, `max_samples="auto"`, `max_features=1.0`, 5-flight smoothing, leading to a validation ROC-AUC of 0.934 ± 0.005. The same grid was also run on the raw-sensor baseline to check whether tuning alone could have reduced it (*see Figure B6*).

LSTM Autoencoder: Feature Representation

We defined the same 12 physical sensors as the Isolation Forest method as our features. Similar to our Isolation Forest method and work done by Sánchez et al. (2026), a key preprocessing step we took in order to eliminate noise caused by shifting flight conditions to affect our results was to conduct operating-condition regression-based normalization. We trained a Multilayer Perceptron (MLP) regression using only healthy engine data to predict values for each of the 12 physical sensors given the four operating conditions (i.e., altitude, Mach number, throttle angle, and inlet temperature). The MLP regressor's goodness-of-fit was validated using the R^2 score (R^2 = 0.997). Normalized residuals are then calculated by subtracting the condition-dependent predictions from the actual sensor values as shown below:

$$residual = actual\ sensor\ value - \hat{f}_{healthy}(alt, Mach, TRA, T2)$$

As our normalized sensor residuals were in their original physical units, which have varying numerical ranges, we globally standardized them across each of the datasets so that their means are centered at zero and their variance is scaled to unit magnitude.

Finally, we transformed the standardized sensor residual values for each of our time-series datasets into overlapping temporal sliding windows of a fixed length of 10 time-steps.We ensured that these sliding windows were created per-engine so that the time steps from one engine unit's operational period would not bleed into another unit's degradation trajectory. These overlapping per-engine sliding windows were then transformed into 3D tensors of shape `(Batch Size, Window Sequence Length, Number of Sensor Features)` to be fed into the LSTM autoencoder to learn how engine sensor behavior and degradation evolves over time.

LSTM Autoencoder: Hyperparameter Tuning & Exploration

For our LSTM autoencoder, we conducted hyperaparameter tuning using a manual grid search across different structural and optimization parameter combinations. We chose to keep two foundational parameters fixed: `WINDOW_SIZE = 10` and

`BATCH_SIZE = 64`. While the `WINDOW_SIZE` defines the number of consecutive time steps used to form each temporal sliding window, `BATCH_SIZE` specifies the number of sliding window sequences that are grouped together and processed simultaneously during each gradient update step.

We chose to use a fixed sliding window sequence length of 10 time-steps because we felt it was sufficient to capture immediate sensor changes and short-term degradation trends. In choosing this window size, we also considered how we had downsampled the original raw N-CMAPSS data at different rates depending on the engine flight condition (`'Fc'`):

- `'Fc'== 1` (1 sample per 76 seconds): A window size of 10 will capture 12.7 minutes of engine operation
- `'Fc'== 2` (1 sample per 127 seconds): A window size of 10 will capture 21.2 minutes of engine operation
- `'Fc'== 3` (1 sample per 180 seconds): A window size of 10 will capture 30 minutes of engine operation

By factoring in these downsampling rates, we believe that a 10-step sliding window sequence provides a reliable temporal background across varying flight regimes. We also chose a fixed standard batch size of 64 because it represents a robust standard in time-series deep learning models, and it has worked well for us in regards to model optimization stability and processing speed.

In addition to our fixed window and batch sizes, within our grid search, we explored three key hyperparameters to optimize our model's performance:

- Hidden Size (`hidden_size` = [64, 128]): Determines the dimensionality of the LSTM encoder/decoder recurrent layers
- Dropout (`dropout` = [0.2, 0.3]): Controls the fraction of units randomly deactivated between LSTM layers during training to prevent overfitting
- Learning Rate (`learning_rate` = [0.001, 0.0005]): Establishes the optimizer step size taken during gradient descent convergence.

For each hyperparameter combination, the model was trained for a maximum of 200 epochs (`epochs = 200`) using a standard Adam optimizer and a `L1Loss` (Mean Absolute Error) criterion function for the reconstructed sensor residuals. We also incorporated early stopping to prevent overfitting by halting training once healthy engine validation loss plateaued. We chose a stricter early stopping patience of 5 epochs (`patience = 5`) to curb excessive training and save compute time and found that all hyperparameter combinations converged and triggered early stopping way before the maximum of 200 epochs.

Ultimately, after each hyperparameter combination was trained until early stopping was triggered, we determined the optimal model configuration to be the one that exhibited the highest ROC-AUC validation metric.

Unsupervised Evaluation

The models never see labels, but `hs` allows us to check whether high scores line up with unhealthy flights. Because the dataset codes `hs = 1` as healthy, we flipped it (`y = 1 − hs`) so that unhealthy flights are the positive class, which is what ROC-AUC, PR-AUC, and recall should measure.

1. **ROC-AUC (Primary):** The chance that a random unhealthy flight scores higher than a random healthy one. This does not require a threshold. Instead, it compares scores, which is what both unsupervised methods produce.
2. **PR-AUC:** Precision across all recall levels for the degraded/unhealthy engine state class. Degraded/unhealthy engines constitute the majority class (as they make up 65% of time-steps), so random scores already earn PR-AUC ≈ 0.65. This is why we report this next to this baseline. For the same reason, we don't report accuracy: labeling every flight "unhealthy" would have already scored about 65%.
3. **Recall, False-Positive Rate (FPR), & F2 at a Threshold:** Missing a degrading engine costs more than an extra inspection, so our goal was to be recall-focused. We selected F2 because it weighs recall *twice* as much as precision. The threshold is chosen on validation engines (Target: 90% recall), then applied unchanged to test engines.
4. **Detection Delay (only Isolation Forest):** Flights between an engine's first unhealthy flight and its first alarm, which is the most practical reading of "how early" for a maintenance team to intervene.
5. **Correlation with Degradation Progress (only Isolation Forest):** Whether scores keep rising as an engine moves toward failure, which tests the score as a continuous measure rather than a binary flag.

IF: Overall Results Reporting

Table B1. Isolation Forest Feature Construction
One change per row (n_estimators=200, random_state=42)

Model Number	Change	Val. ROC-AUC	Val. PR-AUC	Test ROC-AUC	Test PR-AUC
#1	Baseline: Raw means, 12 physical sensors	0.572	0.729	0.641	0.719
#2	Same as #1, Full 1Hz Data	0.550	0.718	0.636	0.721
#3	Operating-Condition Residuals	0.848	0.872	0.815	0.906
#4	#3 + 5-flight Median	0.897	0.946	0.801	0.907
#5	#3 + Virtual Sensors	0.893	0.911	0.842	0.929
#6	**#3 + Virtual Sensors + 5-Flight Median**	**0.923**	**0.965**	**0.832**	**0.928**

Model #6 was selected on validation. Row 2 shows that downsampling cost no information, and Models #3-6 show that the residual correction made a big difference. With physical sensors only, Model #4 is the comparable choice (test ROC-AUC = 0.80).

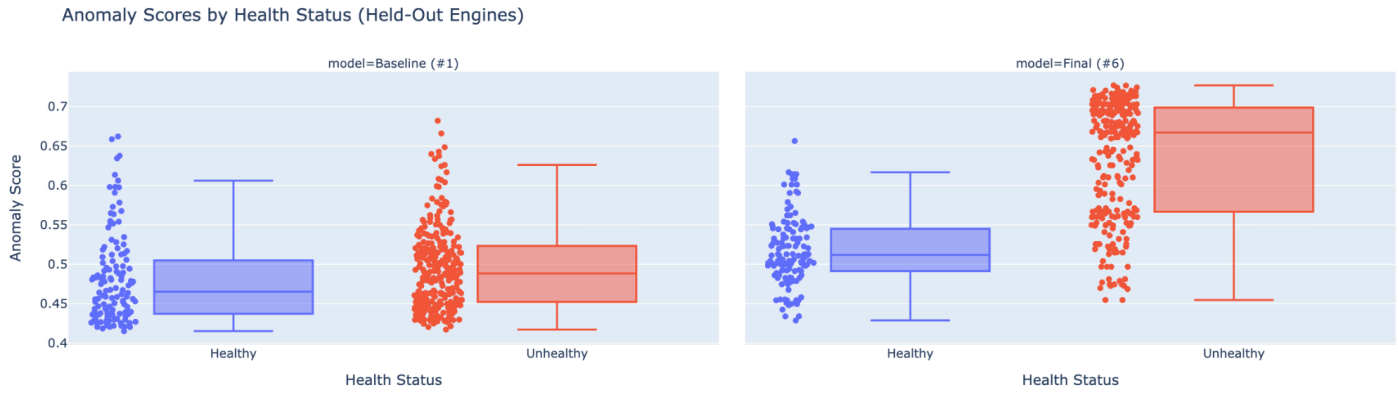


Figure B1. Anomaly Scores by Health Status for 6 Held-Out Engines: Baseline vs. Final Model

(IF Notebook 02, Section 12)

With the baseline, the healthy and unhealthy boxes overlap almost completely (medians 0.465 vs. 0.488). With the final model, the median unhealthy score (0.667) sits above the upper quartile of the healthy scores (0.545).

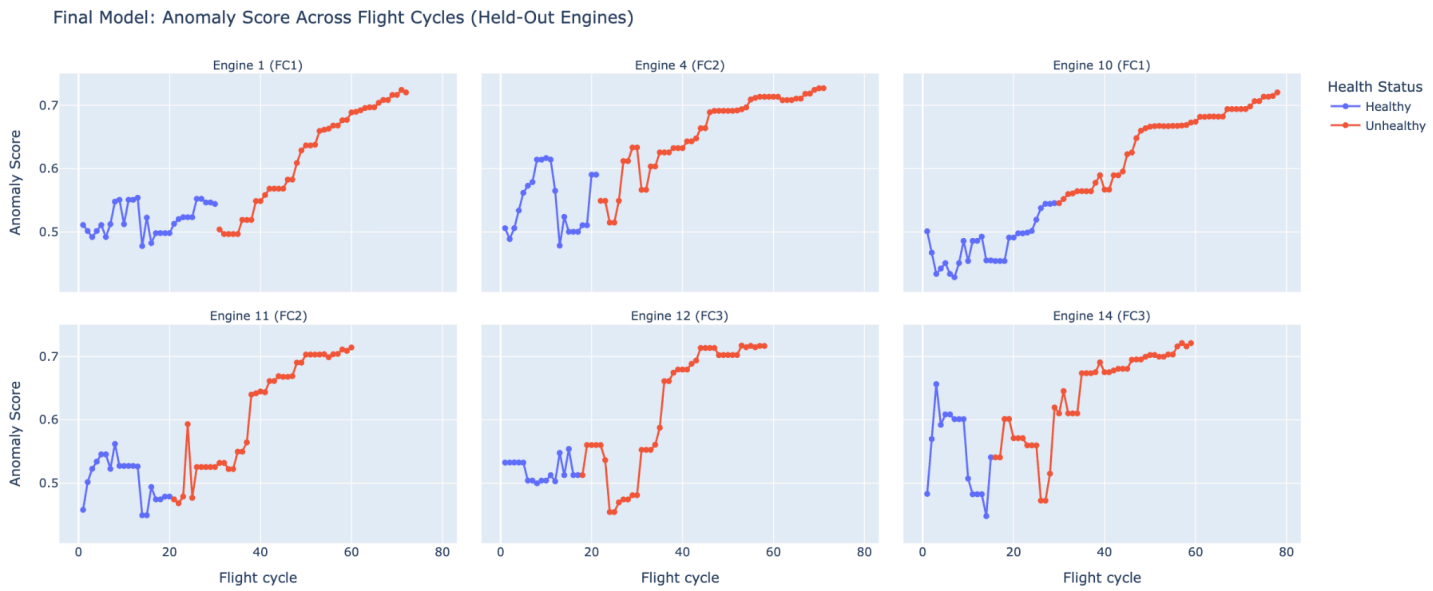


Figure B2. Anomaly Score Across Flight Cycles for Each Held-Out Engine *(IF Notebook 02, Section 13)*

Final model scores stay flat while an engine is healthy, and climb steadily after the transition, which is the degradation trajectory we set out to detect. Baseline scores bounce from flight to flight with no visible relationship to health status.

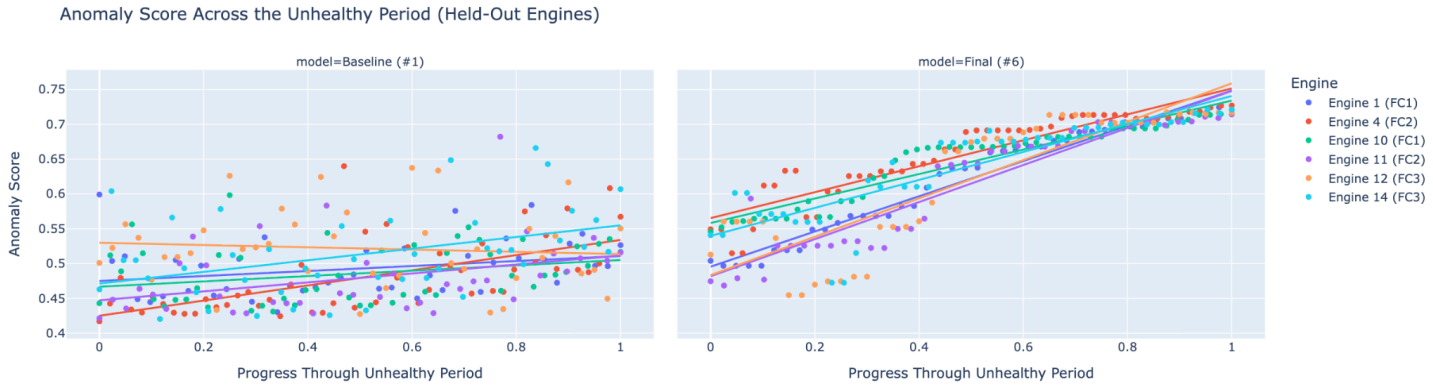


Figure B3. Anomaly Score vs. Progress Through the Unhealthy Period *(IF Notebook 02, Section 14)*

The correlation rises from 0.30 (baseline) to 0.88 (final), and ranges from 0.86 to 0.98 across the 6 held-out engines, so the score behaves like a continuous measure of departure from the healthy, and not simply a binary flag.

IF: Performance by Engine & Flight Class

The residual correction also removed the flight-class-related issue. Each validation and test partition holds one engine per flight class:

Table B2: IF Performance by Engine & Flight Class

Engine (fc)	Partition	Baseline ROC-AUC	Final ROC-AUC
10 (fc1)	Validation	0.740	1.000
4 (fc2)	Validation	0.499	0.918
14 (fc3)	Validation	0.367	0.832
1 (fc1)	Test	0.829	0.856

11 (fc2)	Test	0.493	0.826
12 (fc3)	Test	0.524	0.810

Healthy-flight scores also depended far less on how the plane was flying; their correlation with altitude, Mach, throttle, and inlet temperature dropped from $|r| = 0.31$ to 0.60 (baseline) to 0.25 to 0.33; which reduced, but did not eliminate the correlation.

AE: Overall Results Reporting

Table B3. LSTM Autoencoder Hyperparameter Tuning Results

Model	Hyperparameters			Epochs	Best Validation Loss	Validation ROC-AUC	Validation PR-AUC
	hidden_size	dropout	learning_rate				
1	64	0.2	0.0010	127	0.4113	0.8261	0.9121
2	64	0.2	0.0005	129	0.4288	0.8219	0.9093
3	64	0.3	0.0010	86	0.4318	0.8244	0.9117
4	64	0.3	0.0005	134	0.4349	0.8225	0.9108
5	**128**	**0.2**	**0.0010**	**92**	**0.3506**	**0.8381**	**0.9174**
6	128	0.2	0.0005	99	0.3579	0.8277	0.9116
7	128	0.3	0.0010	115	0.3547	0.8375	0.9171
8	128	0.3	0.0005	103	0.3695	0.8283	0.9124

From examining Table B3, we can see that Model #5 with `hidden_size = 128`, `dropout = 0.2`, and `learning_rate = 0.001` exhibited the highest ROC-AUC validation metric. This is our final model that we use for our visualizations and test evaluation.

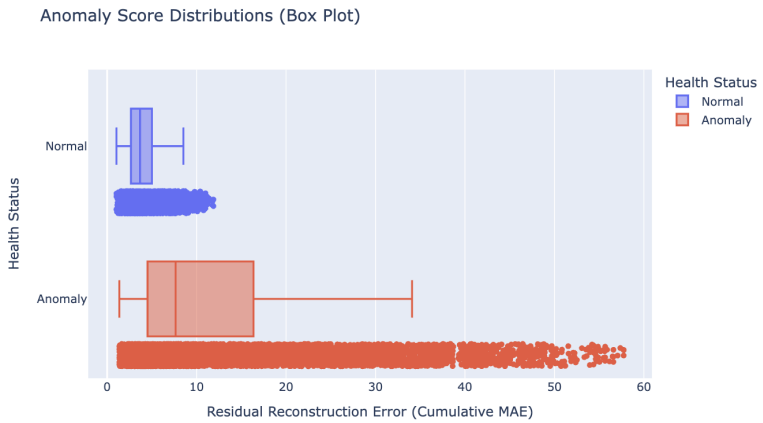


Figure B4. Final Model Anomaly Scores by Health Status for 3 Test Engines *(AE Notebook 01, Section 6.3)*

Our final model exhibits healthy and degraded/unhealthy boxes that do not heavily overlap with cumulative MAE medians of 3.677 and 7.655 respectively. This shows that our LSTM autoencoder assigns higher reconstruction error scores to degraded/unhealthy engine sliding windows, which is what we expected.

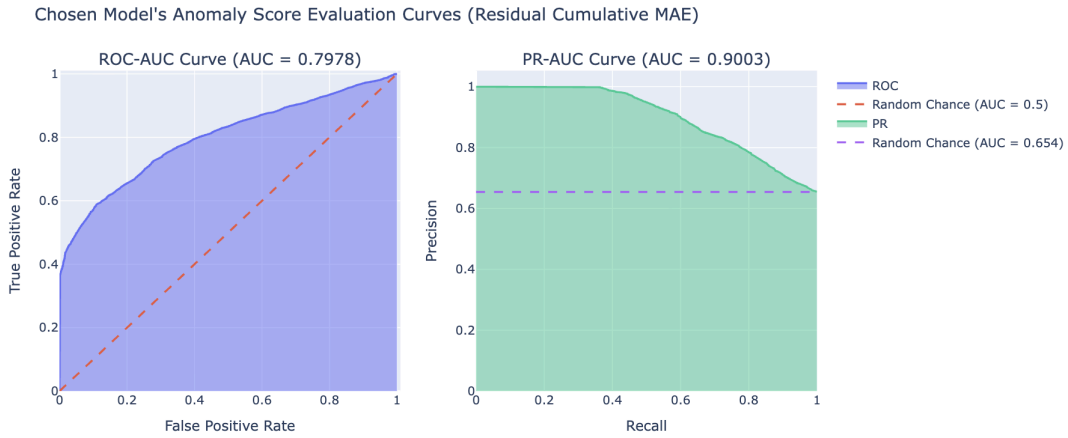


Figure B4. Final Model Test Engine ROC and Precision-Recall Curves with Baselines *(AE Notebook 01, Section 6.4)*
The final LSTM Autoencoder exhibits a strong test ROC-AUC score of about 0.80 and a PR-AUC score of around 0.90, both of which substantially outperform their respective random chance baselines of 0.50 and 0.65 (determined by the test set's 65% degraded/unhealthy engine majority class).These metrics demonstrate that the model is somewhat reliable at distinguishing perfectly healthy/normal engine operations from degraded ones.



Figure B5. Anomaly Score Across Flight Cycles for Each Held-Out Engine, *(AE Notebook 01, Section 6.5)*
The anomaly score trajectories across the test engines (#1, #11, and #12) show the progression of cumulative reconstruction error (MAE) over the 10-step sliding windows. As each of the test engines approach their end-of-life and/or become degraded/unhealthy, the anomaly scores display a sharp, non-linear exponential trajectory upwards. This behavior is expected and showcases our LSTM autoencoder's ability to detect cumulative engine degradation over time.

IF & AE: Overall Results Reporting

Table B4. Best Model from Each Method on the Test Engines (#1, #11, #12)

Method	Features	Test ROC- AUC			Test PR-AUC			Recall	FPR
		Fc = 1	Fc = 2	Fc = 3	Fc = 1	Fc = 2	Fc = 3	Threshold for 90% validation recall	
Isolation Forest	26 Residual Features, 5-Flight Median	0.87	0.79	0.80	0.93	0.91	0.93	0.69	0.015
		Overall: 0.81 ± 0.01			**Overall**: 0.92 ± 0.01				
LSTM Autoencoder	Sliding Windows of 12 Sensor Residuals	0.89	0.80	0.72	0.93	0.90	0.88	0.86	0.56
		Overall: 0.80			**Overall:** 0.90				

Table B4 Note: *Isolation Forest ROC-AUC and PR-AUC are averaged over 5 seeds for the configuration chosen in the grid search. Threshold metrics come from the single-seed Model #6.*

On the three test engines, the final Isolation Forest reaches ROC-AUC score of 0.81 ± 0.01 and PR-AUC score of 0.92 ± 0.01 (mean ± SD over 5 seeds), up from 0.64 and 0.72 for the raw-sensor baseline. Nearly all of the gain comes from the operating-condition correction (the residuals), not from tuning. At the 90% recall target, the threshold chosen on validation (≈ 0.560) catches 91.6% of unhealthy validation flights, but only 69.1% of unhealthy test flights, with just 1 false alarm across 67 healthy test flights. Every test engine is eventually flagged, 1 to 11 flights after onset, and every missed unhealthy flight falls in the first 40% of its engine's unhealthy period.

The final LSTM Autoencoder's overall ROC-AUC score is 0.80, indicating that it has a somewhat strong ability to correctly separate healthy/normal engine states from units that are experiencing degradation as sits 0.30 over the 0.5 random guessing baseline. The model's PR-AUC score is 0.90, which outperforms the 0.65 majority-class baseline by 0.25, signifying that it maintains both relatively high precision and recall. This means that false alarms on healthy engines are more minimal and the model does well at flagging degraded/unhealthy engine occurrences. At our 90% recall target, the threshold chosen on the validation set was (≈ 3.39). This alarm threshold catches about 86% of degraded/unhealthy sliding windows (7,425 true positives) while missing 1,231 instances. Unfortunately, the model does exhibit a considerably high False Positive Rate of around 56% across healthy windows (2,575 false alarms vs. 1,999 true negatives).

Sensitivity Analysis (IF)

Once the features were corrected, the IF's own hyperparameters were less significant than the smoothing window and feature representation.

Table B5: Sensitivity of Validation ROC-AUC Across 192-Configuration Grid

Setting	Values Tested	Spread in ROC-AUC
Smoothing Window	1, 3, 5, 10 flights	0.044
`max_features`	0.25, 0.5, 0.75, 1.0	0.007
`max_samples`	32, 64, 128, auto	0.006
`n_estimators`	100, 200, 500	0.001

Note: Spread = Best Mean □ Worst Mean ROC-AUC across that setting's values

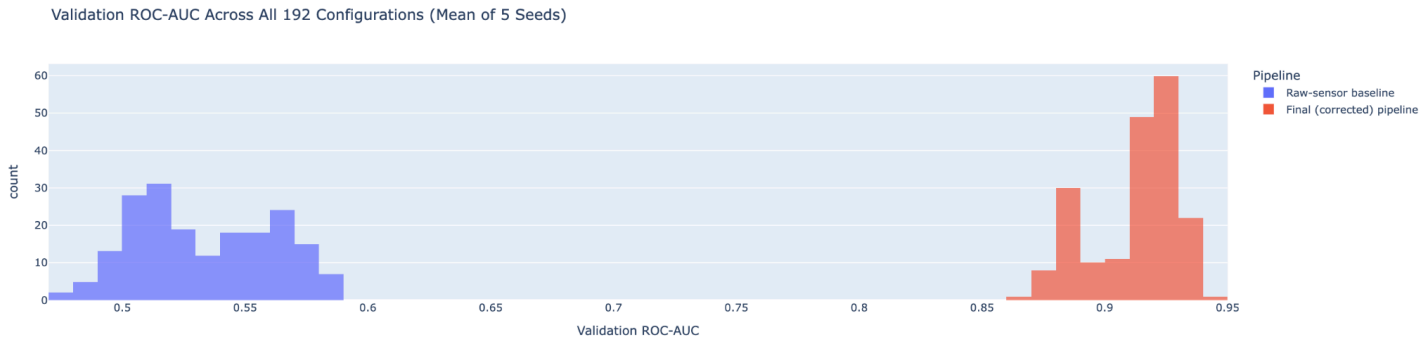


Figure B6. Distribution of Mean Validation ROC-AUC Across All 192 Configurations (Raw Sensor Baseline vs. Corrected Pipeline, *IF Notebook 03, Section 10)*

The two do not overlap; the best baseline configuration (0.589) stays far below the worst corrected one (0.864). The baseline was not under-tuned; its features were dominated by flight conditions.

Correction Model

We also swapped the gradient-boosted correction for the MLP regression-based normalization of Sánchez et al. (2026), the version our LSTM autoencoder uses, while holding everything else fixed.

Table B6: Correction Model Using MLP Regression-Based Normalization on IF *(IF Notebook 03, Section 11)*

Correction Model	Mean R2 on Validation Engines' Healthy Rows	Validation ROC-AUC	Test ROC-AUC
Gradient Boosting	0.9983	0.934 ± 0.005	0.813 ± 0.013
MLP	0.9996	0.931 ± 0.007	0.833 ± 0.007

The MLP fits healthy behavior slightly better on every sensor, but validation ROC-AUC is unchanged (within one seed SD), so gradient boosting is kept. It is important to note that the test difference is reported in Table B6, but it was not used for selection. Removing operating-condition effects matters significantly more than which regression model removes them; and the two Part B methods rest on equivalent preprocessing.

The alarm threshold was a design choice as well (*IF Notebook 02, Section 15.4*). Raising the validation recall target from 90% to 95% lifted test recall from 0.69 to 0.76 and cut the mean detection delay from 5 to 4 flights. The trade-off was that the false-alarm rate (FPR) increased from 0.015 to 0.21. At 99%, recall reached 0.97 while 96% of healthy flights also triggered an alarm.

Discussion

What We Learned from Part A

It was surprising that the engine pressure ratio (EPR) was not at the top of feature importance. It is the ratio of mass airflow to the fan section and mass airflow at the exhaust nozzle. Typically engine performance can be assessed by comparing the degradation over time between exhaust gas temperature (EGT) and EPR. They point toward the condition of the hottest point of the engine, the stator immediately at the outlet of the burner (HPT inlet). For this reason also it is not surprising that HPT efficiencies rank highly.

Staying organized was challenging. There were a lot of hyperparameters in play and the number of features made it more difficult. The Pilgrim class took a long time to get working correctly. It essentially ran MLPs on a list of loss functions. Optimization, learning rate and epochs could be passed in as arguments so that hyperparameters could be applied across all models. It did not change any hyperparameters contained in the MLP structures.

A lot of refining could be done to extend our solution with more time and resources. There were questions about doing different splits. More ablation analysis could be done. Many of the sensor readings are indispensable to engine operation, but could be dropped from the machine learning models. It would be interesting then to narrow down features specifically to each engine section.

What We Learned from Part B

The biggest lesson from **Part B** is that the features mattered far more than the model. A single preprocessing step, such as subtraction of what a healthy engine *should* read under each flight condition, took the IF from near chance to a useful detector, whether gradient boosting or the MLP of Sanchez et al. (2026) performed the correction.

We were surprised to learn that operating conditions strongly affected sensors. We expected operating conditions to add noise; but we did not expect them to explain more than 99.9% of every sensor's variation. In `T48`, degradation shifts T48 by about 9.6°R (≈ 5°C), buried inside roughly 17.7°R (≈10°C) of flight-to-flight variation. After the correction, healthy flights varied by only about 2.0°R (≈ 1°C), so the same shift stands out. Furthermore, we did not expect that tuning hyperparameters would have limited effects on model optimization. We had expected that there would be more considerable changes in our evaluation metrics from using different model hyperparameters, but we found through sensitivity analysis that this was not the case.

A big challenge that we struggled with was limited computational resources when training our model. We found that training multiple hyperparameter configurations of our LSTM autoencoder model was extremely time-consuming and computationally expensive. This made it difficult to make quick changes to our model. As such, in our Google Colab environment, we switched to using a T4 GPU hardware runtime accelerator, integrated PyTorch 2.0+ model compilation (`torch.compile`) to accelerate training speed, limited our hyperparameter search grid to more targeted combinations, and implemented early stopping to prematurely halt training.

If we had more time and additional computing resources, we could extend our solution by expanding our hyperparameter search space for our LSTM autoencoder to evaluate smaller or deeper network structures, larger window sequence lengths, less strict early stopping patience epochs, and so on.

Ethical Considerations

Part A

The study was ethically naive because replacing manufacturer recommended maintenance with anomaly detection methods would be illegal. It is not intended to replace routine maintenance, or manufacturing acceptance testing. All FAA advisory directives are required to be followed and operators should follow advisory circulars to the greatest extent possible. Pilots should be kept informed of system malfunctions and retain the option to override annunciator panel warnings to maximize safety in their best judgement.

Testing that uses anomaly detection should not replace customer requirements. Any feedback from anomaly detection should be an early warning that false positives are more likely, and prompt an investigation.

Ethical misconduct would be the responsibility of those breaking contracts and FAA laws/regulations.

Part B

One ethical concern is the ***consequence*** of false negatives and false positives in a safety-critical maintenance setting. Our Isolation Forest threshold was intentionally recall-focused because the consequence of missing a degrading engine is potentially greater than the cost of an unnecessary inspection. However, the threshold selected to achieve approximately 90% recall on the validation engines detected only 69.1% of unhealthy flights on the test engines. Increasing the recall target also came with a substantial cost: at a 99% validation-recall target, test recall increased to 0.97, but 96% of healthy flights triggered an alarm. Therefore, neither maximizing recall nor minimizing false alarms is ethically neutral; the threshold determines how risk is distributed between missed degradation and unnecessary maintenance.

For this reason, an anomaly score should be treated as ***decision support*** rather than autonomous maintenance decision. A flagged engine could trigger additional inspection or diagnostic review rather than automatically grounding an aircraft or replacing a component. This is consistent with the human-adjudication approach discussed in SIADS 503: that automated systems may support decisions while preserving a meaningful role for human judgment and accountability (Center for Government Excellence, City and County of San Francisco, Harvard DataSmart, & Data Community DC., 2025). In deployment, recall, false-positive rate, and detection delay should be monitored continuously rather than assuming that the threshold selected during development will retain the same operating characteristics.

A second concern is generalizability and the danger of relying on average model performance. Early in Part B, operating conditions (e.g., Mach, altitude), explained more than 99.9% of the variation in healthy sensor readings. The original Isolation Forest therefore tended to identify unusual flight conditions rather than engine degradation. After operating-condition correction, performance improved substantially, but performance still varied across individual engines and flight classes. Thus, a strong overall ROC-AUC may conceal poorer performance under specific operating conditions.

Wallach (2014), while discussing fairness in *social* datasets, warns that optimizing for "good performance on average" could obscure poor performance for smaller subgroups. The same methodological concern could be relevant here even though subgroups are operating regimes rather than demographic groups. In other words, a maintenance model that performs well on average but is unreliable for a particular flight class, engine configuration or operating environment could create ***unequal levels of safety assurance***. We would therefore need to report performance separately by engine and flight class, and test the model under additional operating conditions before deployment rather than relying solely on aggregate metrics.

This concern is especially important because N-CMAPSS is a **simulated** run-to-failure dataset. Performance on heald-out N-CMAPSS engines demonstrates generalization within the simulated environment, but does not by itself establish that the anomaly scores will behave equivalently on physical, real-world aircraft engines. A responsible deployment would therefore require validation against **real-world operational and maintenance datasets** and continued monitoring for distribution shift before the model could be relied upon for operational maintenance decisions. Our models, therefore, should *only* be presented as evidence of a ***potential*** anomaly-detection approach.

Statement of Work

Dillon built the dataset directly from NASA's HDF5 file, set up a hosted database for exploratory analysis of the full 27 GB dataset, engineered the supervised features, and led Part A (supervised learning). Luciana led the Isolation Forest in Part B: exploratory analysis and sensor selection, the operating-condition residual correction, the 192-configuration grid search, and the threshold and detection-delay evaluation. She also rebuilt the dataset from the HDF5 file, organized the GitHub repository, and coordinated the team: scheduling meetings, keeping notes, communicating with our project coach, and setting up the shared Google Drive. Dani led the literature review (including Sánchez et al., 2026), analyzed the dataset to decide how to downsample each flight class, set up the Google Colab compute environment, and led the LSTM autoencoder in Part B: MLP regression-based normalization, sliding-window construction, model design, hyperparameter tuning, and evaluation. All three of us contributed to the written report.

References

Al Bataineh, A., Mairaj, A., & Kaur, D. (2020). Autoencoder based semi-supervised anomaly detection in turbofan engines. *International Journal of Advanced Computer Science and Applications*, *11*(11). https://doi.org/10.14569/IJACSA.2020.0111105

Arias Chao, M., Kulkarni, C., Goebel, K., & Fink, O. (2021). Aircraft engine run-to-failure dataset under real flight conditions. *Data*, *6*(5), pp 1-14. https://doi.org/10.3390/20data6010005

Center for Government Excellence, City and County of San Francisco, Harvard DataSmart, & Data Community DC. (2025). *Ethics and algorithms toolkit (Beta version)*. https://ethicstoolkit.ai [SIADS 503: Module 3]

Cohen, J., Huan, X., & Ni, J. (2023). Fault prognosis of turbofan engines: Eventual failure prediction and remaining useful life estimation. *International Journal of Prognostics and Health Management*, *14*(2). https://doi.org/10.36001/ijphm.2023.v14i2.3486

Jakubowski, J., Stanisz, P., Bobek, S., & Nalepa, G. J. (2022). Anomaly detection in asset degradation process using variational autoencoder and explanations. *Sensors*, *22*(1), 291. https://doi.org/10.3390/s22010291

Liu, F. T., Ting, K. M., & Zhou, Z.H. (2008). Isolation forest. *2008 Eighth IEEE International Conference on Data Mining*, pp 413–422. https://doi.org/10.1109/ICDM.2008.17

Michau, G., & Fink, O. (2020). Unsupervised transfer learning for anomaly detection: Application to complementary operating condition transfer [Preprint]. *arXiv*. https://arxiv.org/abs/2008.07815

National Aeronautics and Space Administration. (n.d.). *Mass flow rate equations*. Glenn Research Center. https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/mass-flow-rate-equations/

PyTorch. (n.d.). *HuberLoss*. https://docs.pytorch.org/docs/2.14/generated/torch.nn.HuberLoss.html

Sánchez, P., Reyes, K., Radu, B., & Fernández, E. (2026). Early fault detection on CMAPSS with unsupervised LSTM autoencoders [Preprint]. *arXiv*. https://doi.org/10.48550/arXiv.2601.10269

Wallach, H. (2014, December 22). *Big data, machine learning, and the social sciences*. Medium. https://hannawallach.medium.com/big-data-machine-learning-and-the-social-sciences-927a8e20460d [SIADS 503: Module 4]

Xu, T., Han, G., Zhu, H., Taleb, T., & Peng, J. (2023). Multi-resolution LSTM-based prediction model for remaining useful life of aero-engine. *IEEE Transactions on Vehicular Technology*. https://doi.org/10.1109/TVT.2023.3319377

Appendix

Figure A1: SHAP Feature Importance

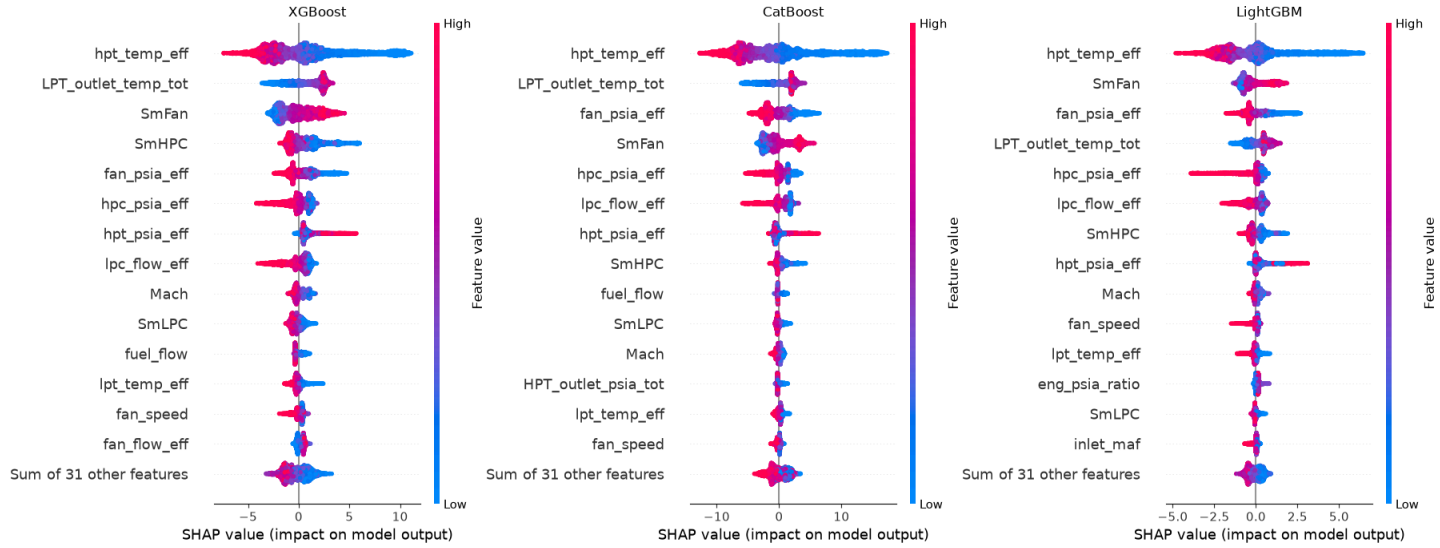


Table A1: N-CMAPSS Supervised MLP Efficiency Metrics

Engine Section	Efficiency Metric 1	Efficiency Metric 2	Notes
Fan	fan_psia_eff	SmFan	Stall margin is loss of blade lift.
Low Pressure Compressor	SmLPC	lpc_flow_eff	
High Pressure Compressor	hpc_psia_eff	fuel_flow	Fuel flow is part of thrust and trim.
High Pressure Turbine	hpt_temp_eff	hpt_psia_eff	Temp efficiency is affected by cooling system capacity.
Low Pressure Turbine	LPT_outlet_temp_tot	LPT_outlet_psia_tot	LPT is a parasitic section, spending gas flow to drive the LPC.

Table 2: N-CMAPSS Variable Mapping

Original	Descriptive name	Original	Descriptive Name
hs	health_status	W21	fan_flow_pps
Fc	flight_cond	W22	LPC_outflow_flow
TRA	throttle_resolver_angle	W25	HPC_inflow_flow
T2	inlet_temp_tot	W31	HPT_coolant_bleed_stageA_flow
T24	LPC_outlet_temp_tot	W32	HPT_coolant_bleed_stageB_flow
T30	HPC_outlet_temp_tot	W50	LPT_outflow

Table 2: N-CMAPSS Variable Mapping

T40	burner_outlet_temp_tot	Smfan	fan_stall_margin
T48	HPT_outlet_temp_tot	Smlpc	LPC_stall_margin
T50	LPT_outlet_temp_tot	Smhpc	HPC_stall_margin
P2	inlet_psia_tot	phi	fuel_flow_to_HPC_outlet_psia_static_ratio
P15	bypass_psia_tot	W48	HPT_outlet_flow
P21	fan_outlet_psia_tot	fan_eff_mod	fan_eff_mod
P24	LPC_outlet_psia_tot	fan_flow_mod	fan_flow_mod
P30	HPC_outlet_psia_tot	LPC_eff_mod	LPC_eff_mod
Ps30	HPC_outlet_psia_static	LPC_flow_mod	LPC_flow_mod
P40	burner_outlet_psia_tot	HPC_eff_mod	HPC_eff_mod
P45	HPT_outlet_psia_tot	HPC_flow_mod	HPC_flow_mod
P50	LPT_outlet_psia_tot	HPT_eff_mod	HPT_eff_mod
Nf	fan_speed	HPT_flow_mod	HPT_flow_mod
Nc	core_speed	LPT_eff_mod	LPT_eff_mod
Wf	fuel_flow	LPT_flow_mod	LPT_flow_mod